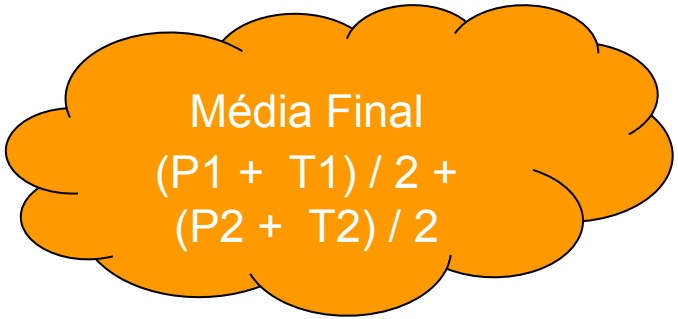
Programação web

Introdução a programação web com JS

Prof. Diego Cardoso Borda Castro

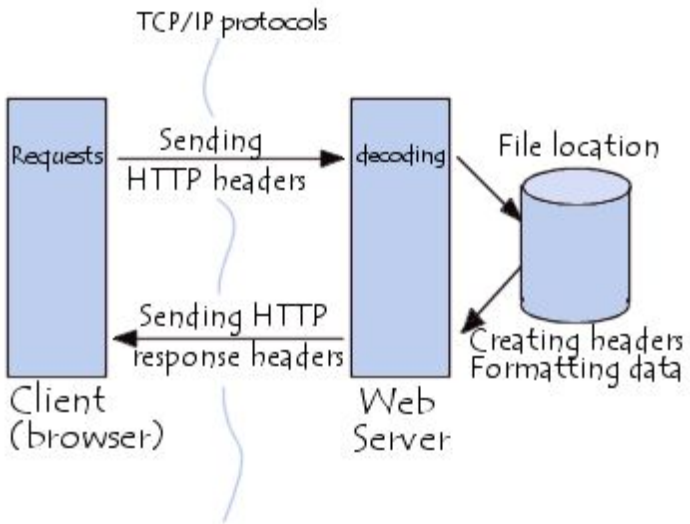
Critérios de avaliação

- Provas individuais e sem consulta
  - Prova 1: 5 pontos
  - Prova 2: 5 pontos
  - Trabalho backend: 5 pontos
  - Trabalho frontend: 5 pontos
  - Prova final e reposição (Matéria Toda)
  - Só será permitido repor uma prova

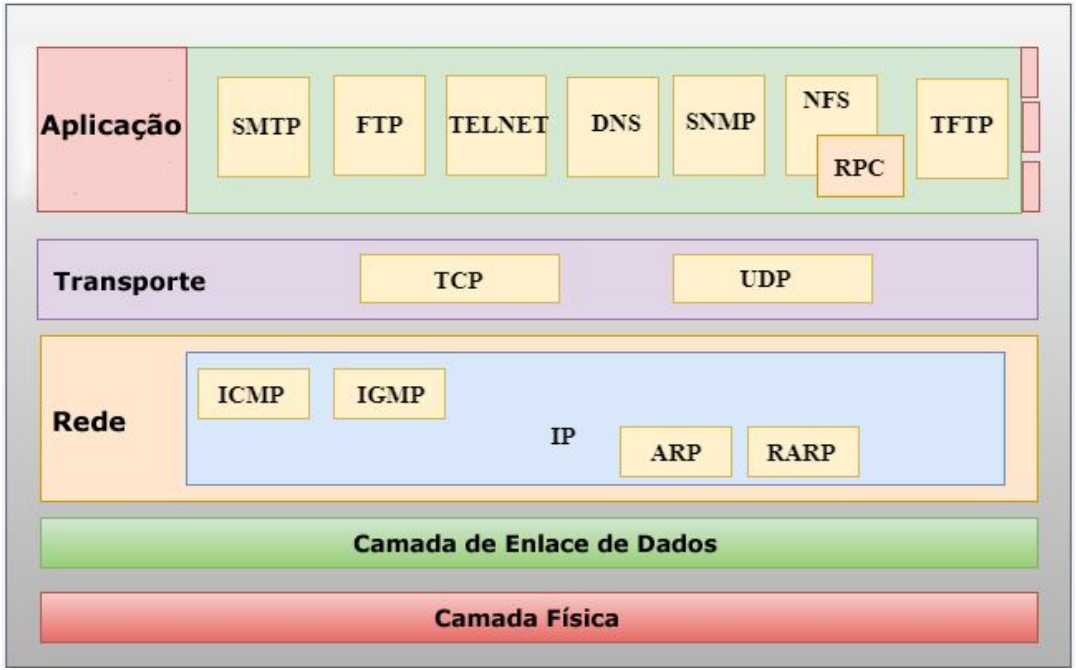


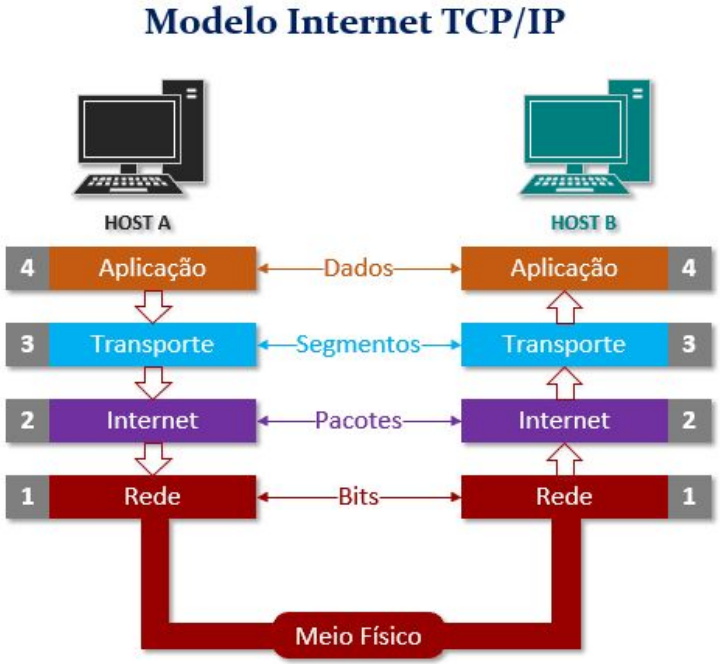
Sistemas Web

- O que são sistemas web?
  - Software hospedado na internet
  - Executados em um browser
  - Funcionam através do protocolo HTTP sobre o protocolo TCP



Sistemas Web





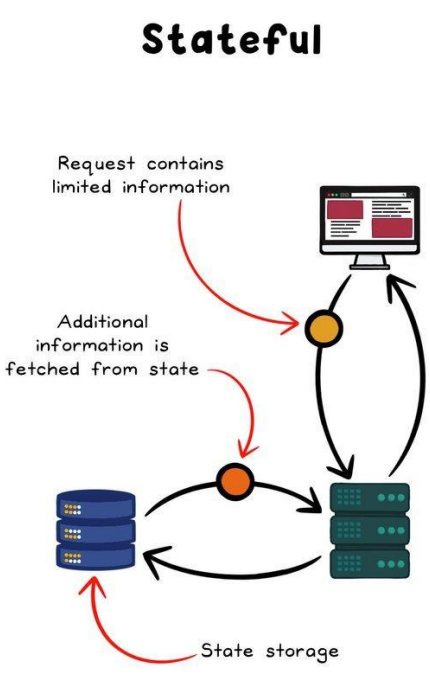
Sistemas Web

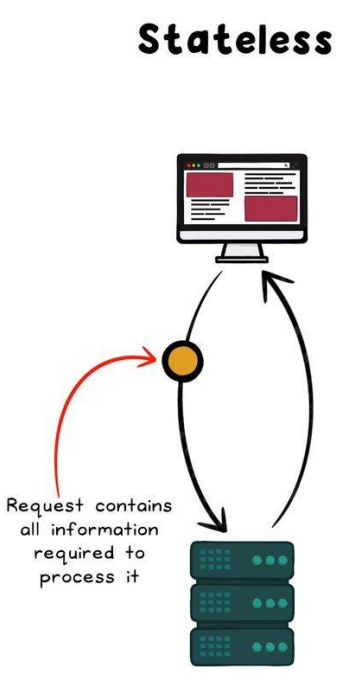
- O que são sistemas web?
  - Transferem bytes em forma de texto.
  - Porta de execução
    - Porta padrão: 80

- HyperText Transfer Protocol
  - Stateless
    - Uma requisição não lembra o que aconteceu antes

Sistemas Web

Mantém o controle do estado da aplicação e permite que os dados possam ser mantido entre diferentes requisições (1)





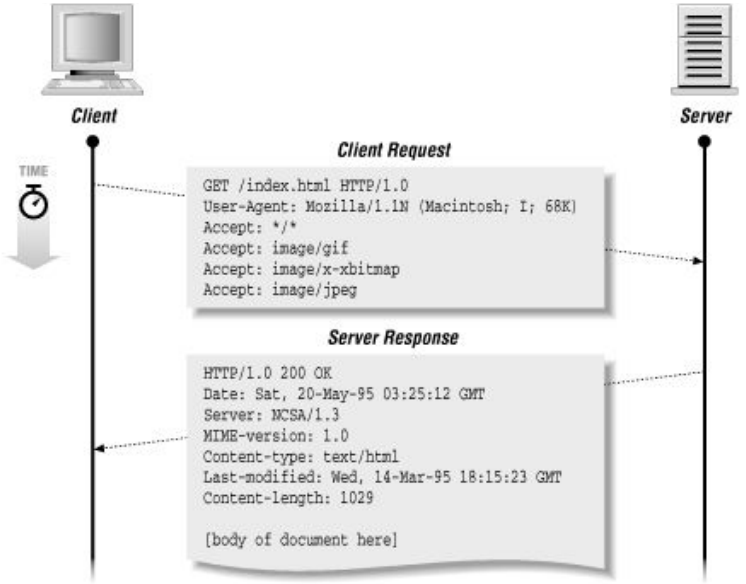
Não há memória (estado) que é mantido pelo programa (1)

Cada interação é tratada de forma independente (2)

HTTP

- Métodos de requisição

- GET
- POST
- PUT
- PATCH
- DELETE
- OPTIONS
- TRACE
- CONNECT
- HEAD

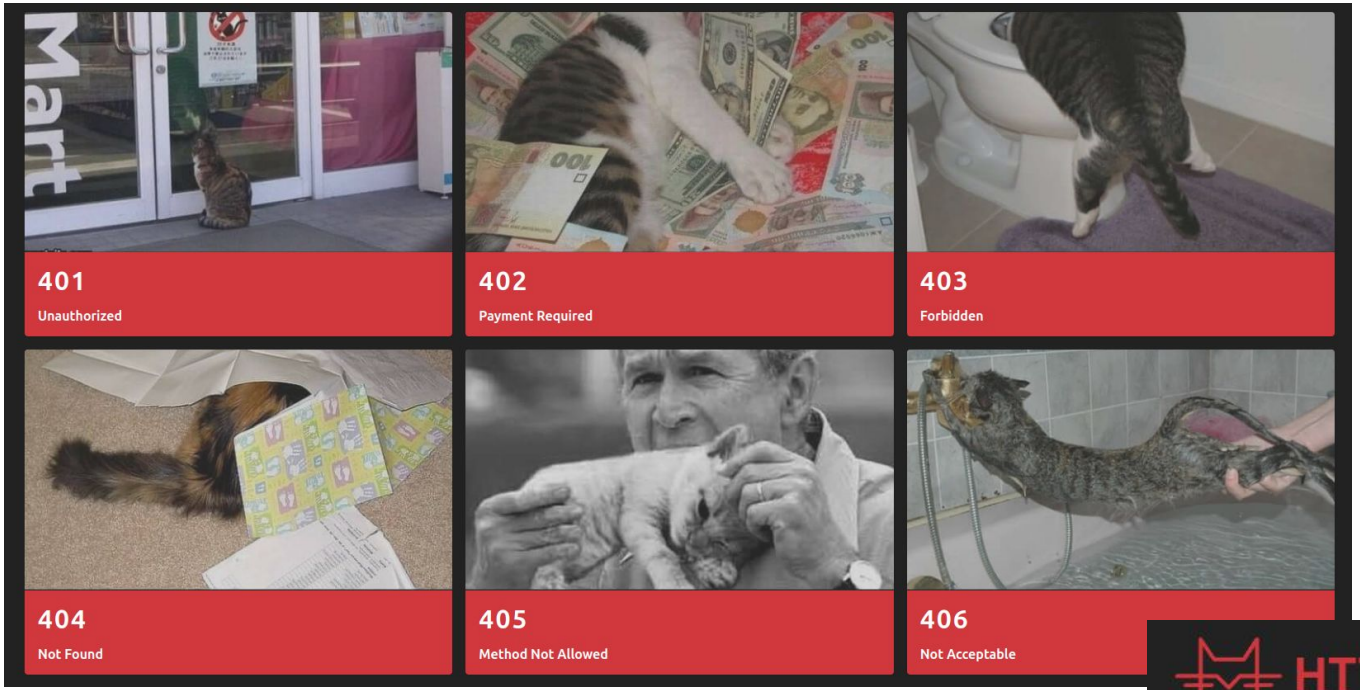


HTTP

- Status HTTP
  - Tipos de respostas
  - Padrão de comunicação
  - http.cat
- Outros
  - 1xx - Informativo
  - 201, 204
  - 400, 409, 418, 422

2xx Success	3xx Redirection	4xx Client Error	5xx Client Error
200 Success / OK	301 Permanent Redirect	401 Unauthorized Error	501 Not Implemented
	302 Temporary Redirect	403 Forbidden	502 Bad Gateway
	304 Not Modified	404 Not Found	503 Service Unavailable
		405 Method Not Allowed	504 Gateway Timeout

HTTP

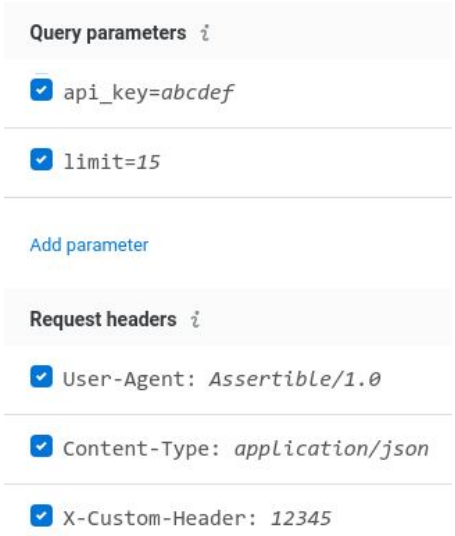


https://http.cat

HTTP

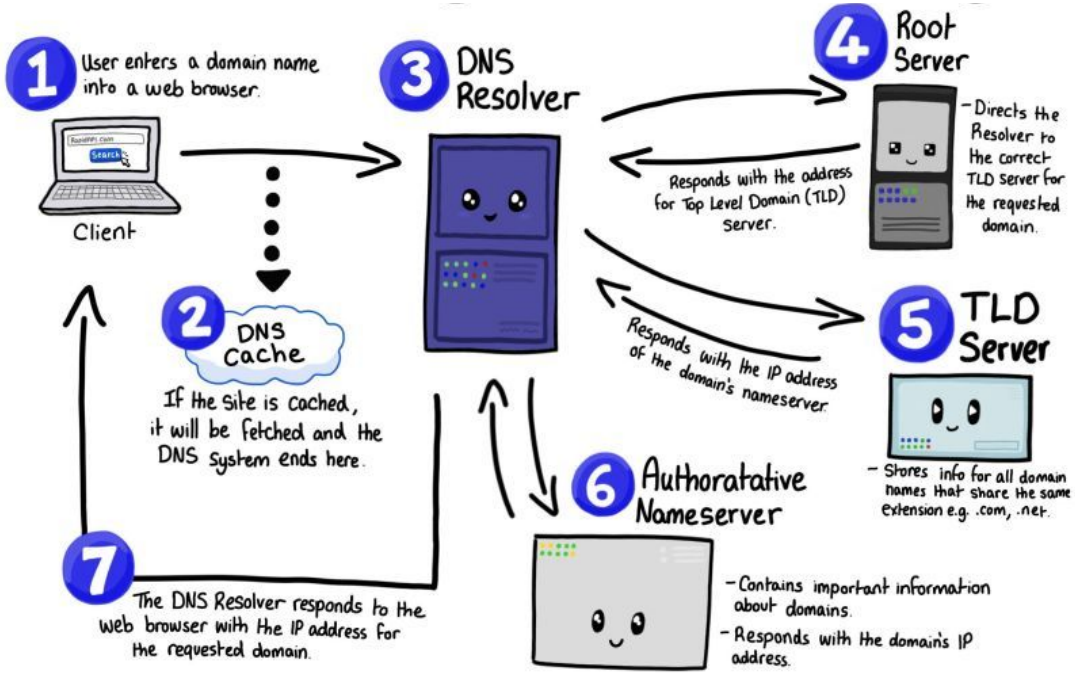
- Parâmetros de requisição
  - Header Params
  - Query Params
    - http://exemplo/rota?valor=1&...
  - Route Params
    - http://exemplo/rota/{id}
  - Body Params

```
{
   "name": "Diego",
   "sobrenome": "Cardoso"
}
```



HTTP

- Domain Name System
- Converte nomes de domínio



Javascript

- Linguagem interpretada
  - Navegadores
  - Runtimes / Interpretadores backend
    - V8
      - Node
      - Deno
    - JavaScriptCore
      - Ban

Javascript

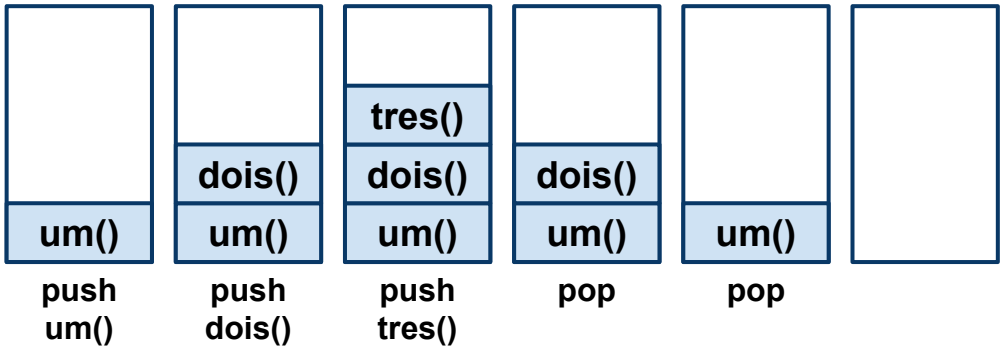
- Front-end
  - Interação com o usuário
  - Telas
  - Funções
    - Formas de interação
      - Ex.: clicks
- Back-end
  - Interação com o banco de dados
  - Regras de negócio mais elaboradas
  - Funções
    - Manipulação de dados
      - Ex.: ORM

Conceitos

- O que é Node.js?
  - Plataforma open-source
  - Execução da linguagem JS no servidor
  - v8 (interpretador) + libuv (i/o async) + módulos
  - Ryan Dahl
    - Barra de progresso
    - Requisições backend
    - Suporte I/O

Características

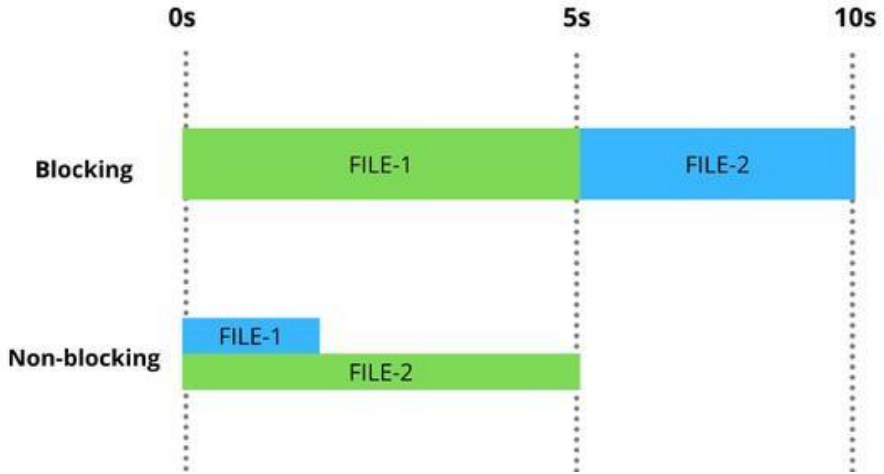
- Arquitetura baseada em eventos
  - Call stack
    - Pilha de funções



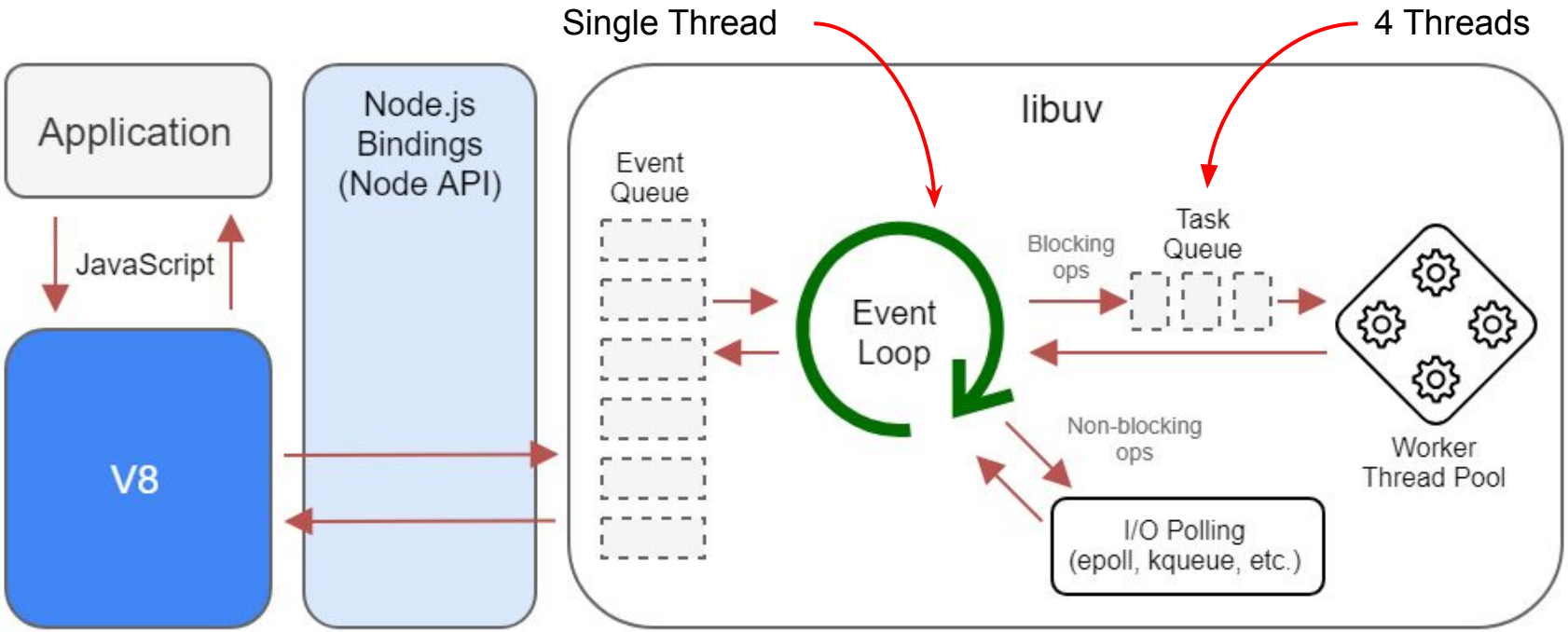

```
1  function um(){
2      dois();
3  }
4
5  function dois() {
6      tres();
7  }
8
9  function tres() {
10     console.log("Aprendendo pilha");
11 }
12
13 um();
```

Características

- Non-blocking I/O
  - Execução paralela
  - Assíncrona
- Módulos próprios
  - Http, Fs, …
- Single Thread
  - Event loop
  - 4 threads padrão



Características



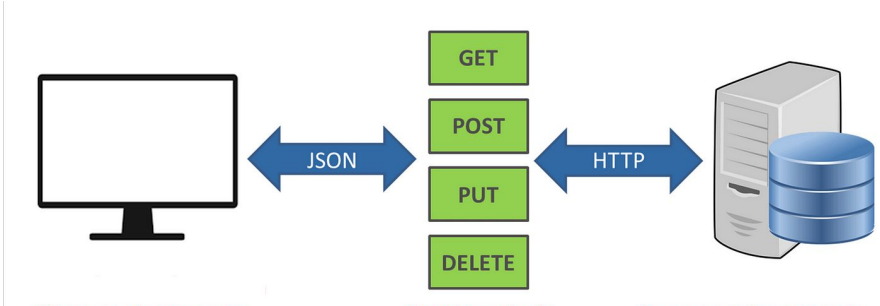
Características

- Gerenciador de pacotes
  - NPM & Yarn
  - Instalar bibliotecas externas
  - Disponibilizar bibliotecas
- Frameworks
  - Express
  - Egg.js
  - Nest.js
  - …

“Abstração que une códigos comuns entre vários projetos de software provendo uma funcionalidade genérica”

API

- Application Programming Interface
  - Interface de Programação de Aplicação
  - Conjunto de especificações para interação
  - Documentação para desenvolvedor
    - Métodos
    - Parâmetros
    - Rotas
    - Ferramentas
      - Swagger
      - Postman
      - Insomnia

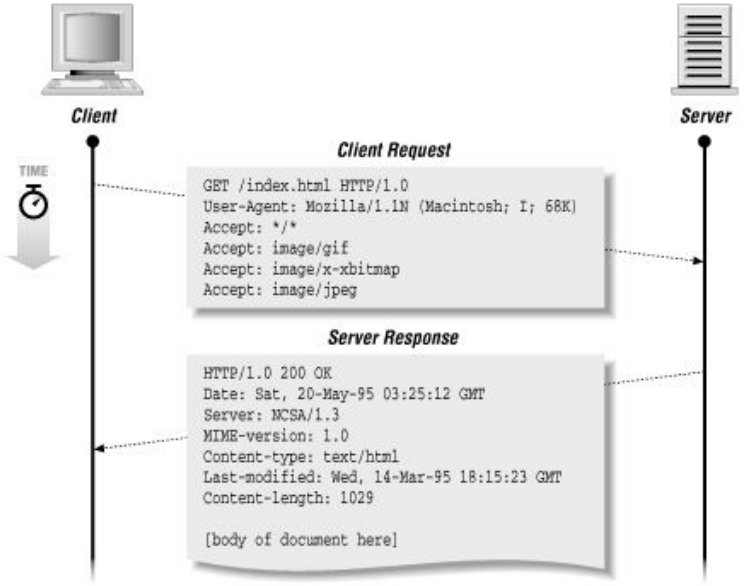


API Rest

- Rest
  - Representational State Transfer
    - Transferência de Estado Representacional
- Modelo de arquitetura
  - 6 regras básicas
    - Cliente - Servidor
    - Stateless
    - Cache
    - Camadas
    - Interface Uniforme
      - Rota
      - Mensagem descritiva
    - Código sob demanda

API Rest

- Métodos de requisição
  - GET
  - POST
  - PUT
  - PATCH
  - DELETE

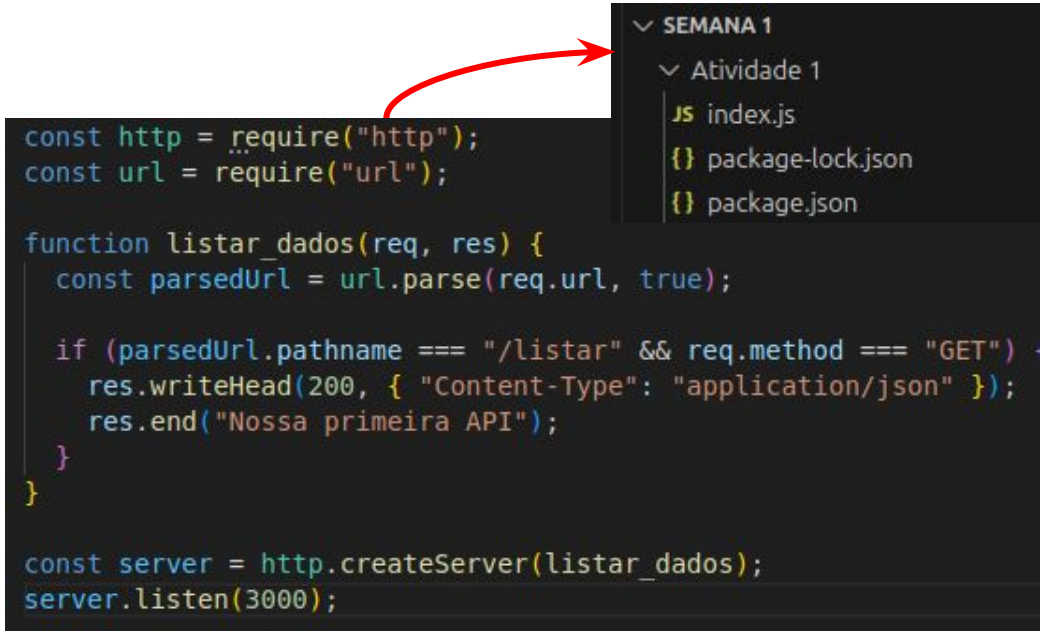


Estrutura do projeto

- sudo apt update
- sudo apt install nodejs
- node -v
- Gerenciador de pacote
  - sudo apt install npm
  - npm install --global yarn

Hello Cefet

- Pasta src
  - Index.js
  - node index.js
- Bibliotecas nativas
- yarn init
- npm init
  - Package.json
    - Arquivo de dependências



Hello Cefet (Express)

- Sem biblioteca nativa
- npm i express

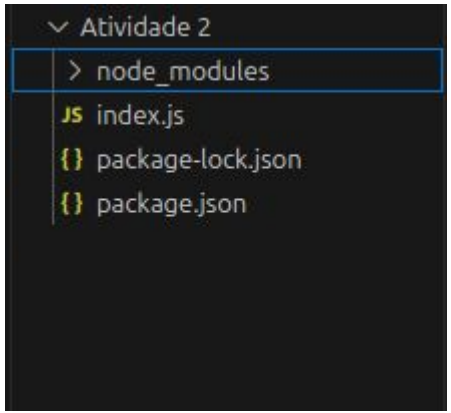
```
Cannot GET /
```

```
1  const express = require("express");
2
3  const app = express();
4
5  app.listen(3000);
```

```
diego@diego:~/Documentos/Aulas/Cefet/Javascript/Semana 1/Atividade 2$ node index.js
```

Hello Cefet

- Sem biblioteca nativa




```
Atividade 2 > JS index.js > listar
1   const express = require("express");
2
3   const app = express();
4
5   app.get("/listar", listar);
6
7   function listar(req, res){
8       console.log("Nossa primeira API 2");
9       return res.json("Nossa primeira API 2");
10  }
11
12  app.listen(3000);
```

```
diego@diego:~/Documentos/Aulas/Cefet/Javascript/Semana 1/Atividade 2$ node index.js
Nossa primeira API 2
```

Auto reload

- node index
- nodemon index
- node --watch index


```
{
  "name": "aula1",
  "version": "1.0.0",
  "main": "index.js",
  "author": "Diego Cardoso",
  "license": "MIT",
  "scripts": {
    "start": "node src/index",
    "dev": "node --watch src/index",
    "nodemon": "nodemon src/index"
  },
  "dependencies": {
    "express": "^4.18.2",
    "nodemon": "^3.0.3"
  }
}
```

Java X Javascript

Java Spring Boot Hello World

```java
package hello;

import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestMapping;

@RestController
public class HelloController {
    @RequestMapping("/")
    public String index() {
        return "Greetings from Spring Boot!";
    }
}
```

```java
package hello;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Node.js Hello World

```javascript
const app = require('express')()

app.get('/', (req, res) => res.send('Hello World from Node.js!'))

app.listen(3000, () => console.log('Listening on 3000!'))
```

Web Server	Apache Tomcat	EXPRESS Js
Platform	Java JVM (Java Virtual Machine)	node js